

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Operator-blocked	1
Bug	Queued	10
Task	In progress	2
Task	Operator-blocked	3
Task	Queued	32
TOTAL		48

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-host routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jack compose env-forward
BOB-068	Bug	Queued	High	RD2-00: unattributed, unreviewed Auto-c mechanism pushing to main
BOB-069	Task	Queued	High	RD2-40: §11.4.238 QA-discovery-channel — ongoing coverage-escape backfill
BOB-070	Bug	Queued	High	RD2-41: pre-build mutation-marker scan false-positive silently defeats entire pre-k gate
BOB-071	Bug	Queued	Medium	RD2-01: guard-forbidden-commands.sh h live reproducible substring carrier false-
BOB-074	Task	Queued	Medium	RD2-07: DDoS-class testing fully absent mandated test-type matrix
BOB-076	Task	Queued	Low	

ATM ID	Type	Status	Severity	Description
				RD2-09: submodules/jackett fork 1 commit behind upstream (informational)
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running the commit rsync/sync mechanism (OPERATOR DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-079	Task	Queued	Medium	RD2-12: Retroactive attributed history no GA-18/21/22/25/26/27 changes (never re-published history)
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code re the substantive Auto-commit diffs
BOB-081	Task	Queued	High	RD2-14: Author CONTINUATION.md Session entry (currently 53 days / 24+ commits behind HEAD)
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable for the four Lava-porting findings (closes GA-14)
BOB-083	Task	Queued	Medium	RD2-16: Regenerate browser_extension/ide/codegraph Status.md + Summary/HTML siblings
BOB-084	Task	Queued	High	RD2-17: Reconcile BOB-008 DB/MD body via the workable-items tool
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/men service v1.0.0 readiness ledger (closes GA-15)
BOB-086	Task	Queued	Medium	RD2-19: Fix BOB-009/BOB-010 evidence backfill item_history for 56 silent closures
BOB-087	Task	Queued	High	RD2-20: Wire docs_chain / commit-seam hook per §11.4.106(F) so DB writes cannot without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Track Items + Status Documents table row-completeness (GA-07 remainder)
BOB-089	Task	Queued	High	RD2-24: RED-first tests for start.sh reload_python/reload_plugins/recreate_scripts (closes test-half of GA-27)
BOB-090	Task	Queued	Medium	RD2-25: HelixQA Challenge entry exercising three start.sh subcommands end-to-end a real compose stack
BOB-091	Bug	Queued	High	RD2-26: Relocate mocked SearchOrchestration tests to unit/ + author real-service replacement (closes GA-14/15/16)
BOB-092	Bug	Queued	Medium	RD2-27: Remove test_live_stack_evidence nmclub SKIP-on-404 fallback + verify live (closes GA-13)

ATM ID	Type	Status	Severity	Description
BOB-093	Task	Queued	High	RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed container (closes runtime half of GA-12)
BOB-094	Task	Queued	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with \$ fault injection
BOB-095	Task	Queued	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py Go-side triangle
BOB-096	Task	Queued	Medium	RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill/resource-exhaustion fault injection
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-098	Task	Queued	Medium	RD2-34: Parametrize 20 hardcoded /Volumes paths in helixqa banks with PROJECT_ROOT (closes GA-23)
BOB-099	Bug	Queued	Medium	RD2-36: Fix guard-forbidden-commands.substring-match false-positive class + add const033-poweroff-signal-triage carrier to EXCLUDE_PATHS
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is -profile go parity still a goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — bit tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, BOB-075 code-reading finding, commit e6162f7): CodeGraph 1.5.0 (up from documented 0 walked into nested-.gitignore-excluded files node_modules and extension/node_modules (32,260 files / 514,456 nodes vs the 2026 baseline of 509 files / 8,906 nodes) instead honoring frontend/.gitignore + extension/.gitignore per git check-ignore Author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh equivalent, e.g. wrapping 'codegraph do -sniff-gitignore-honor' if that subcommand exists, else a real re-index + file-count as with \$11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup and

ATM ID	Type	Status	Severity	Description
				the live nested-.gitignore tree, RED_MOCK asserts the resync stays within the documented baseline order of magnitude. Full evidence in codegraph/Status.md lines 91-118. UPSTREAM FILED 2026-08-18: real upstream repo is github.com/colbymchenry/codegraph (not package @colbymchenry/codegraph, come via package.json + gh repo view; NOT via digital/codegraph, correcting this ledger to original assumption). Issue: https://github.com/colbymchenry/codegraph/issues/1567 (full reproduction evidence: git check-ignore -v verified first-hand; two bounded synthetic attempts up to 63 nested .gitignore files fail against the actual installed v1.5.0 binary; NOT reproduce the blowup in isolation - negative result, root cause not pinned to specific line in either scanning implementation). Draft PR (diagnostic only, not a behavior change) https://github.com/colbymchenry/codegraph/pull/1568 - adds a logDebug() call so a full report can confirm which of the two indexed gitignore-respecting code paths (git-ls-files based vs filesystem-walk fallback) actually fails. Both PRs/issue filed via SSH (Hard Stop from a fork at github.com/milos85vasic/codegraph. Status: In-progress pending upstream maintainer triage/merge - bob's closure of this item still needs the RED/COLOR challenge script (§11.4.115) authored per original scope, independent of upstream response. Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md, meta-deferred during this session's §11.4.140/141 collision resolution, boba commit 136a22c + consensus commits 5ed8c80..e5f2891): §11.4.227(B) anchor-block-integrity (exactly-once head anchor per governance file, byte-identical lockstep across mirrors, no anchor-number collision) is fully specified in constitution but its own named gate CM-ANCHOR-BLOCK-INTEGRITY is explicitly 'gate-code = separate work item, NOT claimed shipped' — the §11.4.140/§11.4.141 double-mandate collision this session's Phase 0 resolved was verified entirely BY HAND (md5 of moved anchor block manual grep counts pasted into a commit message), not by a runnable script. Proposed author a challenge (in this project's challenge
BOB-105	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				scripts/, since this project cannot edit the constitution submodule's own gate-code (this ticket) that greps every governance (CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md/Constitution.md, wherever the project's constitution submodule checks for exposes them) for '### §11.4.NNN' headings occurrences and FAILs on >1-per-file-per or on two DIFFERENT anchor bodies share one NNN, per §11.4.227(B). Do NOT touch constitution-owned files while implementing — the check reads them read-only. Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md RD2-00/BOB entry, docs/GOVERNANCE_AUDIT_2026-08-08_ROU RD2-00): 20 bare 'Auto-commit' commits in this repo's git history (confirmed via git log -oneline -all -grep, e.g. 54e313f/9c8f684/743097a/de9270b/1c36777/41179c2/7c529ca) with no ATM reference and no TDD trail, landing via a git pull fast-forward from a second session with push access to the same remotes (mismatched commit timezone vs the investigating host, per RD2-00 Update). The working-tree-quiescence has no mechanism guard on this path — no gate flags a commit reaching main with a bare/templated message and no ticket citation. Author a §11.4.84 quiescence-check helper (e.g. challenges no_unattributed_autocommit_challenge.sh scans the commit range since the last known good release tag and FAILs on any commit message matching a closed bare/templated pattern (e.g. ^Auto-commit\$, ^sync:) without ATM-NNN or task/PR reference, wired in scripts/pre_build_verification.sh or the §11.4.84 commit-push-all.sh entrypoint. BOB-068 (RD2-00) remains the tracking item for identifying/stopping the source; this item specifically the new automated CHECK. Coverage-escape followup (docs/QA_DISCOVERY_LEDGER.md SCRATCH LOSS-2026-08-18 entry, evidence: .super_sdd/task-phase1a-report.md line 21): the 1a subagent (a1cc331d) discovered at task that 5 source files its brief named as required reads (curriculum_amendment_plan_v1.r ai_curriculum_modules_27_35_extracted
BOB-106	Task	Queued	Medium	
BOB-107	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				3 curriculum_analysis_modules_*.md gaps (analyses) were absent from the session scratchpad — root cause per that subagent's own investigation: the prior producer subagent (ae59171f) hit its session rate limit (a §11.4.147(e) API-quota crash) before writing them. curriculum_amendment_plan_v1.m remains absent from the live scratchpad at this session's re-check. No mechanical check verifies a task brief's declared input files exist and are non-empty before the downstream consumer subagent is dispatched. Author's dispatch precondition helper (project-side orchestration tooling, e.g. a small script conductor runs before Task/Agent dispatches a brief names required input paths) that closed with an actionable 'missing input, respawn the producer' message rather than silently letting a downstream agent proceed on absent evidence and fabricate content unsupported by its named sources. docs/testing/test_type_matrix.md's §11.4.1 type audit found zero scaling-class coverage anywhere in the tree (no scaling-tagged directory, test file, or HelixQA bank distribution growing-dataset/tracker-count/concurrent scale-out from stress-under-burst). Scope at least one scaling dimension, e.g. tracker scale-out in merge search against challenge helixqa-banks/boba-services.yaml's tracker or the qbittorrent-proxy-go -profile go switch with a real measured baseline. docs/testing/test_type_matrix.md's §11.4.1 type audit found UI functional coverage (Playwright) but nothing framed around user accessibility/UX outcomes specifically. Scope axe-core or equivalent accessibility pass on Angular frontend, covering WCAG checks keyboard-nav coverage, and screen-reader labeling. Source inspection across the whole stack (2026-08-18) verified no rate-limit mechanism exists for :7185 (qBittorrent WebUI proxy) (merge search service), or :7189 (boba-jackett) no slowapi/limiter/throttle import in downstream proxy/src/, no rate-limit middleware in qBitTorrent-go/internal/middleware/ (only cors.go + logging.go) or internal/jackett (only auth/cors middleware tests, no rate
BOB-109	Task	Queued	Medium	
BOB-110	Task	Queued	Medium	
BOB-111	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				middleware), and no nginx/reverse-proxy in docker-compose.yml. Candidate remediation documented in docs/testing/ddos_resilience/nginx-in-container reverse proxy with limit_req_zone (most portable, adds a cost to a FastAPI slowapi dependency for the memcached search service; a Gin rate-limit middleware for boba-jackett following the existing internal middleware/ pattern. qBittorrent's own VBR bandwidth-shaping settings were NOT verified to cover request-rate (only bandwidth) - do not assume they close this gap without checking. qBitTorrent-go/internal/jackettapi/health.go:60-63 - HandleHealth makes a synchronous, uncached call to Jackett.GetCatalog() on every single hit to /healthz, with no cache, no distinct timeout, no circuit breaker. Measured evidence (docs/testing/ddos_resilience.md Findings, 2022-07-14, RED_MODE=0, three independent live runs) showed ~98/150 (65%) of health-check requests timed out at 3s under a modest cold-start concurrency burst (10-50 concurrency), recovering to ~100% request once the burst subsided. This is a genuine self-inflicted DDoS amplification caused by an attacker or a mis-configured monitoring probe hitting /healthz too aggressively causing the Jackett-management API's own health-check surface appear down without ever touching Jackett itself. Recommended fixes: cache the Jackett liveness signal with a short TTL, refreshed by a background ticker; add a timeout/circuit-breaker around the GetCatalog call so /healthz itself never blocks past ~250-500ms regardless of Jackett's state. Discovered + scaffolded by BOB-074 (commit ae2b5cb, challenges/scripts/ddos_resilience_challenge.sh); tracked as session task #64 in .superpowers/sdd/progress.md prior to this DB filing - this is the canonical, tracked workable-items reference that reference (§11.4.93 SSoT, §11.4.214 recurrence-links-not-mints: no prior BOB existed for this defect, verified by title/description search before minting). wrk is not installed on the development host. ApacheBench (ab) is, and was used as the primary DDoS-resilience-challenge load test. The task brief's 'wrk or ab' fallback path, curl-parallel-loop as the actual per-status
BOB-112	Bug	Queued	High	
BOB-113	Task	Queued	Low	

ATM ID	Type	Status	Severity	Description
BOB-114	Task	Queued	Medium	assertion mechanism (more precise than aggregate summary) and ab's own output as supplementary real-tool evidence. Instructions (e.g. via the project's setup.sh or a documentation apt/build-from-source recipe) so future local DDoS challenges have a modern HTTP benchmarking tool with latency-percentile histograms available out of the box, matching the brief's stated primary preference. challenges/scripts/ddos_resilience_challenges-self-validate mode currently only ships a golden-bad fixture for the crash-resistance detector (per §11.4.107(10)/§11.4.201). The limiting detector has no matching golden fixture proving it would actually FAIL a no-rate-limit-enforced artifact, so an unvalidated rate-limit detector could silently pass a bogus absent rate-limit deployment. Add a syntactic fixture (e.g. a stub server that never returns 429/503 under burst) and assert the detector correctly reports the absence, closing the validation gap for this detector class. search.py:1329 (_search_rutracker passsword login path) still classifies a no-session-cookie login result as error_type=auth_failure with message 'rutracker login returned no session cookie — likely CAPTCHA wall or credential failure'. The adjacent code comment (lines 1244-1248, same function) already states the FACT that 'rutracker gates login.php behind CAPTCHA when logins spike... the CAPTCHA is the real blocker' — yet the error classification still hedges with forbidden §11.4.6 vocabulary (likely) and reports the WRONG error_type (auth_failure instead of upstream_captcha_failure) unlike the exact same class of bug already fixed for nnmclub per docs/qa/nnmclub-login-diagnosis-20260616.md (which replaced 'credential failure' with a FACT-based upstream_captcha classification + explanatory comment explicitly citing "This is NOT likely a credential failure"). The rutracker code path could disambiguate the same way nnmclub does (or by reading the login response body for captcha marker, mirroring the existing path used ~20 lines later in the SAME function to parse the search response) but currently discards this opportunity and ships a guess. Found during §11.4.6 bluff audit (docs/qa/task-bluff-audit). Direction: apply the same upstream_captcha
BOB-117	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				reclassification + FACT-based message u nnmclub, or add a captcha-marker check login response before falling back to a he message. README.md line 28 ships a green/succe colored shields.io badge ‘python tests-58 passing’, and line 333 repeats ‘585+ test real, non-destructive verification this ses (timeout 90 nice -n 19 ionice -c 3 .venv/b python -m pytest tests/ -collect-only -q) o 5235 tests, not 585 — roughly 9x higher, badge is either wildly stale or was never machine-derived (violates §11.4.259 CM- MACHINE-DERIVED-SOURCE + §11.4.6 guessing). The README’s own §11.4.259 provenance footnote (lines 40-57) explici discloses that the tests/vitest/plugins/me scan/license badges were ‘carried forward unverified this session (out of this task’s and directs readers to docs/TESTING.md current authoritative per-suite counts’ — docs/TESTING.md contains NO occurrence ‘585’ or ‘182’ anywhere, so the cited authoritative source cannot actually corr the badge. The frontend vitest badge (18 passing) is also suspect: a source-level g it()/test() calls in frontend/src/**/*.spec.ts ~371, roughly 2x the claimed 182 (lower confidence proxy than the pytest collect- count, but consistent with the same stale pattern). Found during a §11.4.6 bluff au (docs/qa/task-bluff-audit/). Fix direction: real badge-computer (README already f this as an owed §11.4.197 gate-code item regenerates the badge from a real collec count, and add the actual per-suite count docs/TESTING.md so the cited authoritat source is real. docs/MERGE_SEARCH_DIAGNOSTICS.m ~128 states ‘Default: 1 (all trackers expo dead ones filtered by DEAD_PUBLIC_TRACKERS)’ for ENABLE_DEAD_TRACKERS. This is factu wrong: download-proxy/src/merge_servic search.py:1032 reads os.getenv(‘ENABLE_DEAD_TRACKERS’, (default string ‘0’) and docker-compose.y sets ENABLE_DEAD_TRACKERS=\0 (also default 0). The sibling doc docs/ DEAD_TRACKERS_EXPLAINED.md corre
BOB-118	Bug	Queued	High	
BOB-119	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				states 'With ENABLE_DEAD_TRACKERS (default): 24 public trackers active, 14 ex — confirmed against the current DEAD_PUBLIC_TRACKERS frozenset (14 entries, names match exactly). MERGE_SEARCH_DIAGNOSTICS.md's s default is the one document that disagree the source of truth, and could mislead an operator into believing dead trackers are to end users by default when they are ac filtered out by default. Found during a \$1 bluff audit (docs/qa/task-bluff-audit/). Fix direction: correct 'Default: 1' to 'Default: MERGE_SEARCH_DIAGNOSTICS.md to search.py/docker-compose.yml.